

Simple Structure – MS DOS

Simple-structure operating systems have no well-defined architecture and are small, simple and limited in functionality. Their interfaces and functional levels are not clearly separated, which makes them less reliable. Because of this lack of separation, a failure in any user program can cause the entire system to crash.

- No well-defined structure; small and limited.
- Interfaces and functionality levels are not separated.
- MS-DOS is an example of this structure.
- Applications can directly access basic I/O routines.
- A user program failure can crash the entire system.

An Application could pass the resident system, Device drivers, more dangerously could pass all the layers. Can also issue a direct INT13h call to ROM/BIOS.

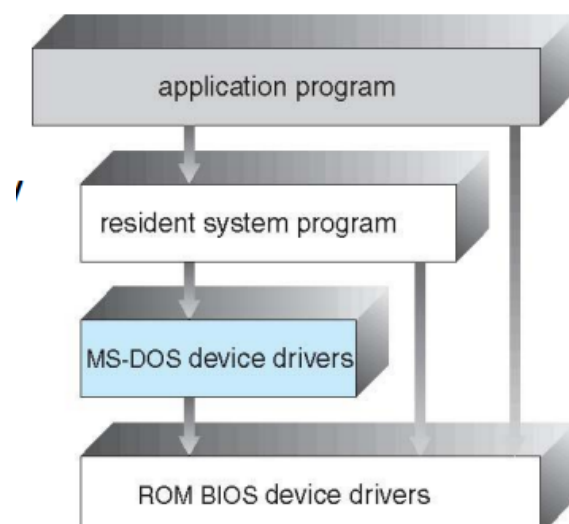
The direct access may have allowed a virus or bug program to corrupt the disk.

Cases: When any Apps needs directly access to Hardware/BIOS

- Performance: Critical Operations (Games and Damascene)
- Accessing unsupported Hardware
- Low Level System Utilities.

Cases When App Follow the Layers:

- Standard File and Console Operations (Like text editors)
- Portability across hardware (Business Applications – Browser)
- Simplicity of Development
- When you a Good Citizen



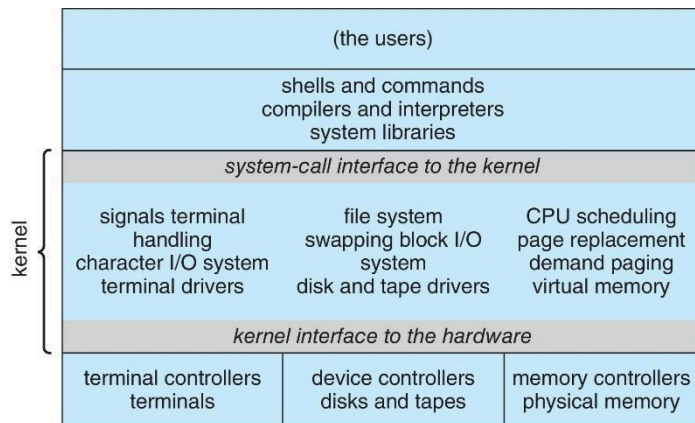
Monolithic Kernel Architecture

Monolithic Kernel Architecture is an operating system design where all OS components (process management, memory management, file systems, device drivers, etc.) run in a single address space in kernel mode.

Real World Example

Imagine a large corporation with no departmental divisions - everyone works in one giant room, sharing everything directly. Fast communication but chaotic if something fails.

Architecture Diagram:



Characteristics

- Single Address Space: All OS services share the same memory space
- Direct Function Calls: Components communicate via function calls (not messages)
- Kernel Mode Execution: Everything runs with highest privileges

Examples:

- UNIX (Original): The prototype monolithic system
- Linux: Most famous modern monolithic kernel
- Windows 9x Kernel: Mostly monolithic design
- Traditional Mac OS (pre-OS X)

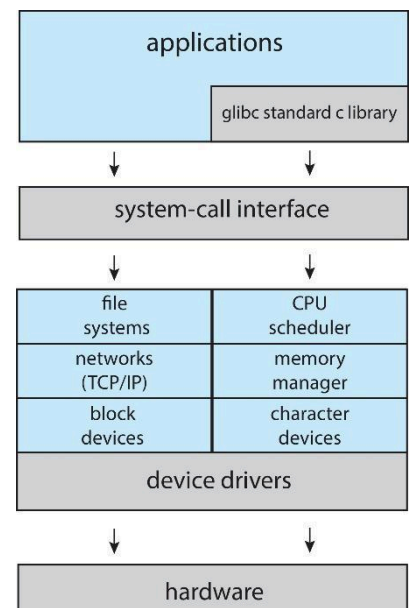
Linux System Structure (Monolithic + Modular)

- Linux is a Monolithic Kernel that behaves like a Microkernel when it needs to.
- Linux is monolithic, but it supports loadable kernel modules (LKMs).

A kernel module is a piece of kernel code that can be loaded and unloaded at runtime, without rebooting the system.

Examples:

- Device drivers (USB, Wi-Fi)
- File system support
- Network protocols

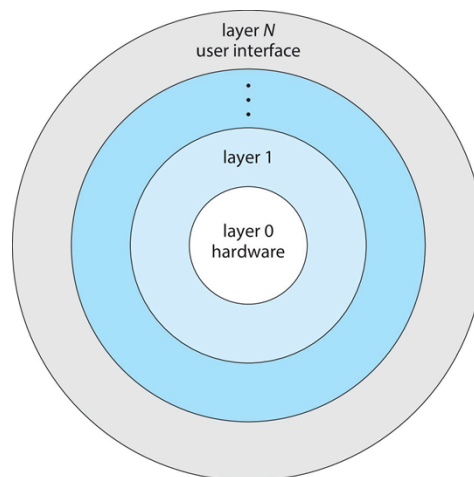
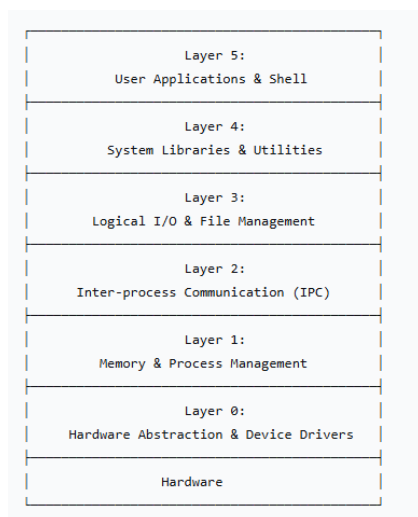


Monolithic Kernel Code Example

```
// Simplified view of kernel function calls
void sys_open_file(char *filename) {
    // All in kernel space:
    security_check(filename);      // Security subsystem
    inode = filesystem_lookup(filename); // File system
    fd = allocate_file_descriptor(); // Process management
    driver = get_device_driver(inode); // Device management
    driver->open(inode);           // Direct driver call
    return fd;
}
```

Layered Approach

In this approach there is a hierarchy of abstraction layers, where each layer uses services of the layer below it and provides services to the layer above it.



Characteristics

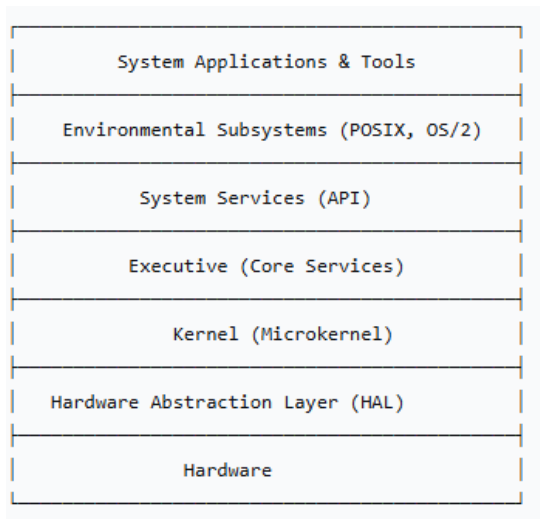
- Unidirectional Dependency: Layer N only depends on Layer N-1
- No Circular Dependencies: No layer depends on higher layers
- Well-Defined Interfaces: Each layer exposes clean APIs
- Information Hiding: Implementation details hidden within layers

Examples

Dijkstra's THE System (1960s):

- Layer 0: Processor allocation, multiprogramming
- Layer 1: Memory management (512K drum)
- Layer 2: Operator-process communication
- Layer 3: I/O management
- Layer 4: User programs
- Layer 5: System operator

Windows NT (Original Design)



Advantages:

- Modularity: Each layer can be developed/tested independently
- Debugability: Problems isolated to specific layers
- Maintainability: Changes in one layer don't affect others
- Formal Verification Possible: Can prove correctness layer by layer

Disadvantages

- Overhead of Layering: Each layer adds function call overhead
- Inefficient Communication: Data may pass through unnecessary layers
- Context Switching: Between layers may require mode switches

Microkernel Architecture

Microkernel Architecture follows the minimalist principle: move as much functionality as possible from the kernel into user-space processes, leaving only essential mechanisms in the kernel.

Real Life Example:

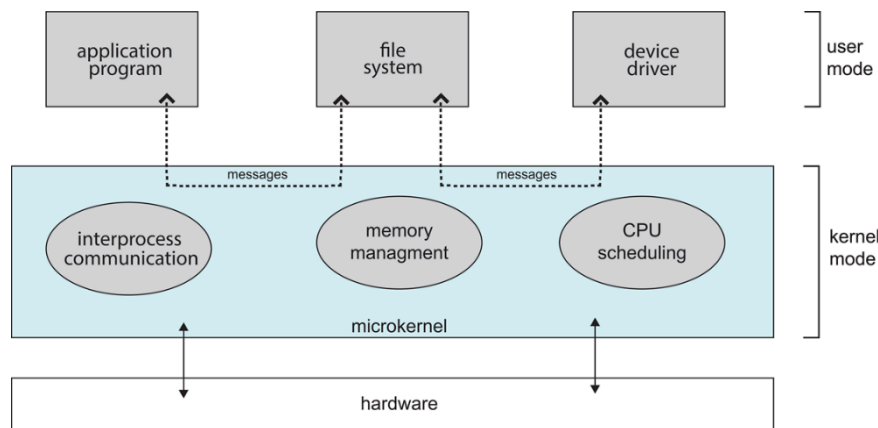
Government and its Departments

Government = Microkernel

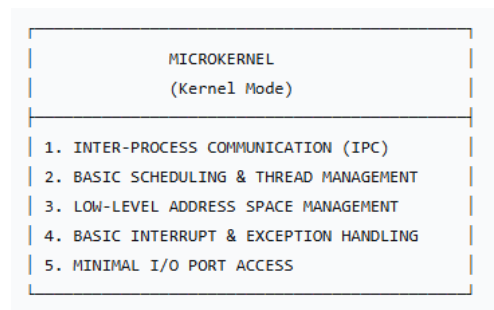
Departments are independent

- Transport, Tax, Police

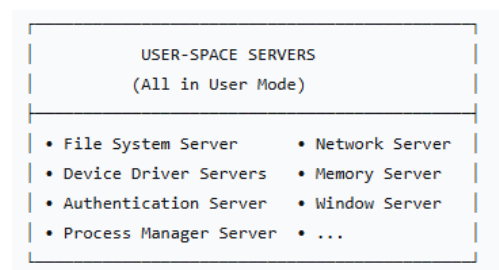
Think of a government where the government (microkernel) only handles essential services (defense, law), while specialized departments (user-space servers) handle everything else (education, transportation, healthcare). Departments can fail independently without collapsing the government.



What Stays in the Kernel (Minimal Set):



What Stays in the User Space?



Some OS Examples having Microkernal Architecture

- Mach (1985): Carnegie Mellon University - Most influential microkernel
- L4 (1990s): Jochen Liedtke - Focus on performance
- QNX (1980): Commercial real-time microkernel OS
- Minix 3 (2005): Andrew Tanenbaum's teaching/research OS
- seL4 (Secure Embedded L4)
- Zircon (Google Fuchsia)
- Genode

How It Works: Message Passing Paradigm

Microkernels use system calls primarily to implement message passing and basic kernel mechanisms, while higher-level OS services communicate using IPC rather than direct system calls.

“Microkernels minimize system calls and use them mainly to support IPC.”

IPC (Inter Process Communication)

- Synchronous Message Passing: Sender blocks until receiver responds
- Asynchronous Message Passing: Sender continues after sending
- Capabilities: Token-based access control to resources

Sequence of message passing:

1. App calls read() -> Traps to kernel
2. Kernel sends message to File Server
3. File Server validates request
4. File Server sends message to Disk Driver
5. Disk Driver performs actual I/O
6. Results travel back through chain
7. Kernel returns to application

Modular (Loadable Kernel Modules)

Loadable Kernel Modules are pieces of code that can be dynamically loaded into and unloaded from the kernel at runtime without rebooting the system. They extend kernel functionality while maintaining a modular architecture.

Characteristics

- Dynamic: Load/unload without system restart
- Privileged: Execute in kernel space with full privileges
- Shared: Can export symbols for other modules/kernel to use
- Persistent: Can maintain state across operations

Module Metadata Section

- /* Linux example */
MODULE_LICENSE("GPL"); // License (GPL, MIT, proprietary)
MODULE_AUTHOR("Name"); // Author information
MODULE_DESCRIPTION("Brief"); // Module description
MODULE_VERSION("1.0"); // Version info
MODULE_ALIAS("alias_name"); // Alternative names
MODULE_DEVICE_TABLE(...); // Device matching table

Example

```
#include <linux/init.h> /* Required for the macros */
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Cypher <cypher@example.com>");
MODULE_DESCRIPTION("A simple example of kernel metadata");
MODULE_VERSION("0.1");
/* Example for a PCI device table */
static struct pci_device_id my_ids[] = {
    { PCI_DEVICE(0x1234, 0x5678), },
    { 0, }
};
MODULE_DEVICE_TABLE(pci, my_ids); //Through this function, Kernel will know to
automatically load it when the device is attached.
```

Module Lifecycle Management

- **Loading Process**

- User Command: *insmod module.ko* or *modprobe module*
- Space Allocation: Kernel allocates memory
- Relocation: Address resolution and symbol linking (Resolving external references - connecting module's calls to actual kernel functions/data)
- Init Call: *module_init()* function executes
- Registration: Module registers with appropriate subsystems

```
Module.ko (on disk)
  ↓ (loaded into memory)
Relocatable Binary
  ↓ (address resolution)
Fix absolute addresses
  ↓ (symbol linking)
Resolve external references
  ↓
Executable Module in Memory
```

- **Unloading Process**

- Reference Check: Ensure no active users
- Cleanup: *module_exit()* function executes
- Deregistration: Remove from kernel subsystems
- Memory Free: Release allocated resources

Advantages of LKMs

- Reduced Kernel Size: Load only needed functionality
- Dynamic Updates: Patch/update without reboot
- Development Efficiency: Test modules without rebuilding kernel
- Customization: Enable/disable features per system
- Third-party Support: Allow proprietary drivers

Limitations

- Security Risks: Malicious modules can compromise system
- Stability: Poorly written modules can crash kernel
- Performance Overhead: Module loading/unloading overhead
- Compatibility: Module must match kernel version/configuration

Examples:

- Linux (Monolithic + Modular)
- FreeBSD (Monolithic + Modular)
- Solaris/Illumos (Modular kernel with DTrace integration)

Hybrid Structure

A hybrid kernel combines elements of both monolithic and microkernel architectures, aiming to achieve the performance benefits of monolithic kernels with the modularity and reliability of microkernels. It moves some non-essential services out of kernel space but keeps critical components in kernel mode for efficiency.

Key Principles:

- Performance-Critical Code runs in kernel space (For Speed)
- Non-Essential Services run in user space (for stability and security)
- Controlled Communication between kernel and user components
- Selective Modularity - not everything is modular

Architectural Components

- **Kernel Space Components**
 - Process/Thread Schedule
 - Memory Management Unit (MMU)
 - Interrupt/Exception Handling
 - Basic IPC Mechanisms
 - Hardware Abstraction Layer (HAL)
 - Security/Authorization Core
- **User Space Components**
 - File System Drivers
 - Network Stack (partially)
 - Device Drivers (many)
 - GUI Subsystem
 - System Services (log, time, etc.)

Key Characteristics

- High performance (less IPC overhead than microkernels)
- Modular design
- More stable than monolithic kernels
- Easier maintenance and updates

Examples of Hybrid Operating Systems

- Windows NT / Windows 10 / 11
- macOS (XNU kernel)
- Combines Mach (microkernel) + BSD (monolithic elements)
- BeOS